



TECHNOLOGY
SOLUTIONS^{UK LTD}
part of **HID**

Android SDK v2.8.0

Developer Guide

25th April 2024

www.tsl.com

Design • Development • Manufacture

Table of Contents

Introduction	4
TSL ASCII 2.0 Protocol History	4
API Introduction	5
Command Parameters	5
Executing a Command and Response Handling	5
Connecting to TSL UHF RFID Readers	6
Setting up an Android project	8
Working with ePop_Loq® and USB OTG connected Readers	9
Sample Applications	11
Overview	11
Inventory Sample Application	12
Inventory Sample Code Overview	12
Trigger Sample Application	14
Trigger Sample Code Overview	14
ReadWrite Sample Application	16
ReadWrite Sample Code Overview	16
Licence Key Sample Applications	18
LicenceKey	18
LicenceKeyUserApp	19
Licence Key Sample Code Overview	20
ConfigureBarcode Sample Application	21
ConfigureBarcode Sample Code Overview	21
ReaderManager Sample Application	23
ReaderManager Sample Code Overview	23
TagFinder Sample Application	24
Tag Finder Sample Code Overview	24
HFLFSupport Sample Application	26
HF LF Support Sample Code Overview	26
S3SensorTagReader Sample Application	28
Basic Operation	28
Sensor Reading Display Area	29
S3SensorTagReader Sample Code Overview	29
About TSL	31
Contact	31
About HID	31

Document Revision History

ISSUE	DATE	AUTHOR	NOTES
1.0	25/4/2024		First Issue

Introduction

The Android v2.0 SDK builds on the foundations of the ePop-Loq® SDK to provide a single, unified, SDK that developers can use to produce Android Apps that connect to TSL ASCII 2 Readers using either ePop-Loq®/USB OTG or Bluetooth connections. The v2.0 SDK now provides a much leaner implementation designed to integrate with existing applications with minimal code while maintaining a high level of backwards compatibility to previous SDKs.

All Sample Projects have been fully updated to use the new [ReaderManager](#) & [Reader](#) classes and the new [DeviceList](#) activity to provide a Reader selection interface.

A new Sample Project, [Reader Manager](#), has been added that shows just the code needed to manage a connection to the Reader, whether it be over ePop-Loq®/USB or Bluetooth®. This project is accompanied by a step-by-step guide describing how to create the App starting from an Android Studio Basic Activity project.

TSL ASCII 2.0 Protocol History

ASCII 2.0 is a recreation of the ASCII protocol Technology Solutions originally created for their range of UHF readers. The aim of the original protocol was to enable rapid testing of UHF commands from a terminal command prompt connected to a Technology Solutions UHF reader. As more readers implemented the protocol the ASCII protocol became useful to support multiple platform types without having to support the full binary API.

The main objectives of the ASCII 2.0 protocol were to maintain the ease of use for experimenting at the command prompt but to add more structure to the commands and responses to make it easier to command from an application. This has been achieved with the following:

- A defined start sequence to every command (e.g. “.iv”)
- A consistent way to pass parameters to a command
- Simple framing of all commands and responses with a header and line terminator
- Signalling the termination of a response with an empty line
- Signalling the error for a command with a return code “ER: xxx” or “OK”

The ASCII 2.0 protocol can be implemented in most modern languages very simply. Once a connection is established to the reader a command line is sent to the reader. Lines of response are then read from the reader until an empty line is received signalling the end of the response. The line preceding the empty line will start “OK:” or “ER: xxx” indicating whether the command executed successfully.

All ASCII commands start with a period ‘.’ followed by two characters to identify the command e.g. “.iv” for inventory. The command is terminated with an end of line (Cr, Lf or CrLf). The rest of the command line can contain parameters with or without values. A parameter starts with a minus ‘-’ symbol followed by one or more characters to identify the parameter then followed by the parameter value.

“.iv -x” perform an inventory, reset the command parameters to default

An ASCII response is a sequence of lines terminated by an empty line. The each line has a two character header followed by a colon ‘:’ the remainder of the line is the value corresponding to the header.

“ME: this is a message” is a message response as part of an ASCII response.

An ASCII response always starts with the command started “CS:” line and ends with either “OK:” if the command completed successfully or “ER: xxx” if it failed, xxx represents an error code. If the “ER:” is sent is may also be preceded by an “ME:” line with a human readable error.

API Introduction

The purpose of the API for ASCII 2.0 is to provide a library of commands that enable a developer to rapidly build the commands to send to the reader and also to interpret the responses.

There is Javadoc documentation in the `AsciiProtocol Library` project to browse all the classes, interfaces and members.

Command Parameters

The `IAsciiCommand` interface represents a command to be sent to the reader and there are classes that implement this interface for all of the common commands. These command classes have properties to set the parameters to send to the reader.

All the parameters for a command are optional, where a parameter is not specified the reader uses its cached value of the parameter. Every parameter has a “Not specified” value, this is the value of the parameter that should be used when that particular parameter should not be sent to the reader.

Where commands have parameters, they have a reset to defaults ('x') parameter to reset all parameters to their defaults in the reader before executing the command. Note executing the command includes updating any parameter on the command line to the specified value. Commands also have a read parameters parameter ('p') to read the current value of all the commands parameters.

Commands can also have a take no action parameter ('n'). When specified the command parameters can be read or modified without performing the actual command (e.g. an inventory can be configured without actually performing the inventory, the inventory can then be performed by another inventory command without specifying any parameters).

Executing a Command and Response Handling

An instance of `IAsciiCommandExecuting` is used to execute an `IAsciiCommand` with the `executeCommand` method. `AsciiCommandExecutorBase` is the base implementation of `IAsciiCommandExecuting`. It executes a command by sending a command line to a reader using its `Send` method. Responses from the reader are processed by the `ProcessReceivedLines` method.

`AsciiCommander` is an implementation of `IAsciiCommandExecuting` that extends `AsciiCommandExecutorBase` and is used to send lines to and receive lines from a `Reader`. Received lines are passed to the `processReceivedLines` method. We recommend using a single instance of `AsciiCommander` that exists for the life of your App – `AsciiCommander` provides `AsciiCommander.sharedInstance()` for this purpose. At the start of an Activity, you ensure that the shared existence exists by calling the `createSharedInstance` method() as follows:

```
// Ensure the shared instance of AsciiCommander exists
AsciiCommander.createSharedInstance(getApplicationContext());
```

An ASCII command can either execute synchronously or asynchronously. The `IAsciiCommandExecuting` has a chain of `IAsciiCommandResponders` which get called in sequence to handle each line that is received in the response. Each responder has the opportunity to mark the line as handled so no further responders get notified. There is a `SynchronousDispatchResponder` which when inserted into the chain can relay responses to the executing command. For this to work the command must also implement a responder to capture its own response. This is achieved with the `AsciiSelfResponderCommandBase` class. When a command is its own responder the command executes synchronously and `executeCommand` blocks until the response to the command has been received. For a command to execute synchronously its `synchronousCommandResponder` should be set to itself (this).

If an executing command does not have its `synchronousCommandResponder` set then `executeCommand` will return as soon as the command is sent and the `SynchronousDispatchResponder` in the responder chain will have no action. It is then down to other responders in the chain to handle the response to the command. The command has executed asynchronously.

As each command implements its own responder the commands themselves can be placed into the responder chain to capture the responses of other instances of the same command that are passed to `executeCommand`. In addition, custom responders can be implemented like the `LoggerResponder` (inserted at the start of the chain to log but not handle all responses).

Connecting to TSL UHF RFID Readers

A Singleton of *ReaderManager* is accessible through the *ReaderManager.sharedInstance()* property but this instance must be created prior to use as it requires the *ApplicationContext* – this is done using *ReaderManager.create(Context)*.

The *ReaderManager* needs to know when the Activity is paused and resumed so add into the Activity's lifecycle events as follows:

Activity.onPause()

```
ReaderManager.sharedInstance().onPause();
```

Activity.onResume()

```
ReaderManager.sharedInstance().onResume();
```

ReaderManager.getReaderList() provides an *ObservableReaderList* of *Reader* objects representing TSL readers that are currently "known" to the terminal. "Known" readers are those that are:

- Connected using an ePop_loq® case
- Placed in the docking cradle and connected using a USB OTG cable (terminal requires OTG support)
- Paired via Bluetooth®

Note: The v2.0 SDK only supports a single connected reader.

To recognise Readers that are already attached when the App launches add the code below to the Activity's *onResume()* method:

```
ReaderManager.sharedInstance().updateList();
```

ObservableReaderList provides events for added, updated and removed *Reader* instances - this can be used to track the physical connection and disconnection of a reader and when the *Reader* has changed transport.

The *ReaderManager* caches information about Readers that it discovers when that information becomes available (once a connection has been made to it and its *DeviceProperties* have been set). This means that how a Reader identifies itself (*getDisplayname()* and *getDisplayInfoLine()*) can change on the first few times it is used.

It is possible for the same physical Reader to, initially, appear in the *ReaderManager* list twice once identified by its Bluetooth® pairing and secondly as an unknown ePop-Loq®/USB device. Once a connection has been made to the *Reader* over the USB transport the list will be adjusted to represent the device as a single *Reader* instance (and appropriate list change events are fired). When the same Reader is next encountered the *ReaderManager* will use its cached knowledge and represent it as a single *Reader* in the list.

The *Reader* class provides the methods to *connect()* and *disconnect()* from the physical reader and is used by *AsciiCommander* to communicate with the physical reader. Each Reader can potentially communicate over one or more transports so there are two *connect()* methods:

- *connect()* – Connects using USB, if available otherwise over Bluetooth®
- *connect(TransportType)* – Connects on the transport specified only.

The *Reader* remembers which transport type was last successfully connected in the *Reader.getLastTransportType()* property.

By default, *ReaderManager* will automatically connect to any ePop-Loq®/USB Reader as it is attached/discovered. To prevent this behaviour, use *setWillAutoConnectUsbReaders(false)*. Be aware that the physical Reader will switch to communicate over USB in preference to Bluetooth® when plugged in over a USB connection, even if the App chooses not to talk over the USB transport – in this state, the Bluetooth® messages will be ignored.

Once a *Reader* instance has been discovered call the *AsciiCommander.setReaderProperty()* to notify *AsciiCommander* of which *Reader* to use.

AsciiCommander provides notifications when the *ConnectionState* of the *Reader* changes (as it did in previous versions of the SDK). Since it is now possible to communicate over different transports to the same Reader, multiple notifications occur when an USB connection is made to an already Bluetooth-connected Reader. By default, first the

Bluetooth connection is disconnected (raising the state changed notification), then the USB connection is connected (raising another the state changed notification). When *AsciiCommander's ActiveTransport* changes, an 'abort' error is passed through the responder chain to ensure that any currently running synchronous commands will terminate appropriately.

Each *Reader* instance holds its own *DeviceProperties* and is initially null - this property is automatically populated by *AsciiCommander* for its current *Reader* once that *Reader* has successfully connected.

AsciiCommander still provides a *DeviceProperties* property for the current *Reader* as in previous versions of the API – this method returns the current Reader's properties when connected or *DEVICE_DEFAULTS* otherwise.

Setting up an Android project

The following section describes integrating the Ascii Command library into projects built using Android Studio – the Google® supported Android development environment.

The supplied SDK is designed for use with Android Studio and will allow any of the sample projects to be built and tested.

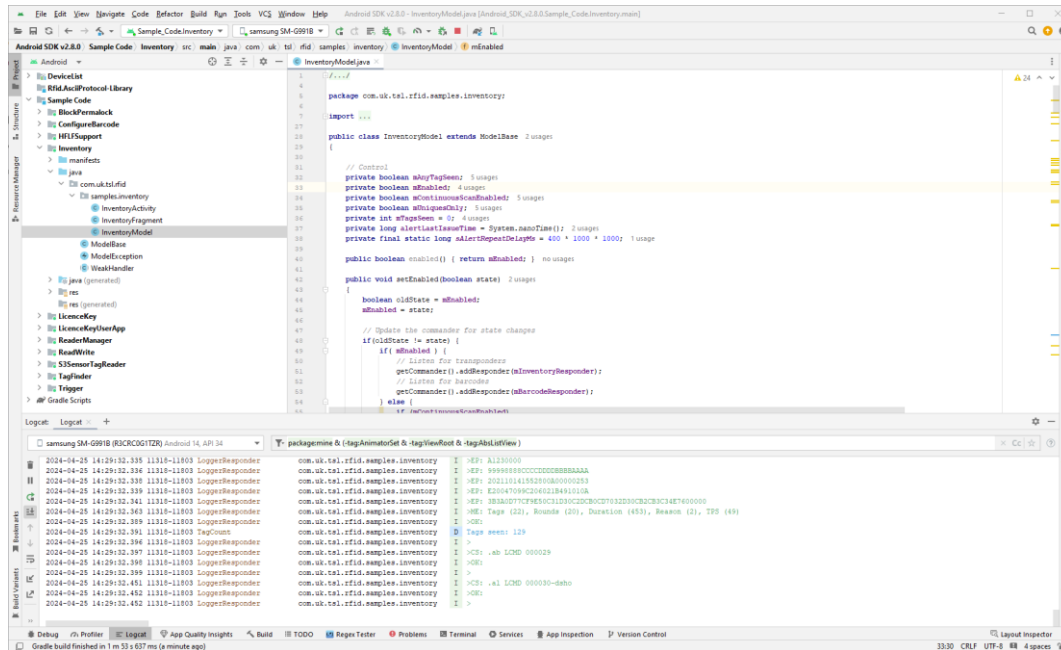


Figure 1 The TSL Android SDK in Android Studio

The Rfid.AsciiProtocol-Library module provides the Rfid.AsciiProtocol library in “.aar” format and the associated documentation.

To add this to your project:

- Copy the Rfid.AsciiProtocol-Library folder to your project folder.
- Add this line to the Settings.Gradle file


```
include ':Rfid.AsciiProtocol-Library'
```
- Run a Gradle Sync.
- The included [Javadoc](#) documentation can be associated with the library using the [Project view](#).
 - Expand the 'External Libraries'
 - Right-click the 'Rfid.AsciiProtocol-Library-unspecified' item.
 - Open the 'Library properties'
 - Click the green +
 - Navigate to the 'Rfid.AsciiProtocol-Library/doc' folder and click 'OK'
- Run a Gradle Sync

- In the Project view, right-click on the module that you want to use the library with and select 'Open Module settings'.
- Ensure that the correct module is selected and then click the Dependencies Tab.
- Use the + button to add a new 'Module Dependency'
- Select the 'Rfid.AsciiProtocol-Library' from the list presented.

You should now have access to the TSL AsciiProtocol API from within your project along with integrated documentation.

Working with ePop_Loq® and USB OTG connected Readers

Android requires that attached USB devices (such as ePop_Loq® mounted devices) are granted permission by the User to be accessed by the App.

The ePop_Loq® RfidAsciiProtocol library will request permissions as necessary but developers can choose two ways that this happens:

- 1) By adding a declaration to the App manifest the App can be launched whenever a reader is attached – the first time this occurs the User can choose to open the App Once or Always. If the User chooses Always then they should not see any permissions dialogs on subsequent reader attachments. Note: sometimes the User may have to OK an 'Allow...' dialog before the 'Open' dialog appears.
- 2) Without the declaration the App will prompt the User for permission each time the App is launched. Although the dialog offers the option of "Use by default for this USB device" in many Android implementations this setting appears, to the User, as having no effect (actually it is due to the Terminal providing a different 'name' for the device each time it connects).

To implement 1) add the following to the AndroidManifest.xml:

In <Activity>

```
<intent-filter>
    <action android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED" />
</intent-filter>

<meta-data android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
android:resource="@xml/device_filter" />
```

Add the following to the Activity to ensure only one instance of the activity is launched if the App is already open and device is attached. Otherwise, a new Activity is created for each time the device is connected.

```
android:launchMode="singleTask"
```

The operation described in 2) is the default behaviour of the library.

Full details about Android USB Host devices can be found here:

<https://developer.android.com/guide/topics/connectivity/usb/host.html>

The next section describes the sample code projects provided to demonstrate the use of the AsciiProtocol Android Library Project.

Sample Applications

Overview

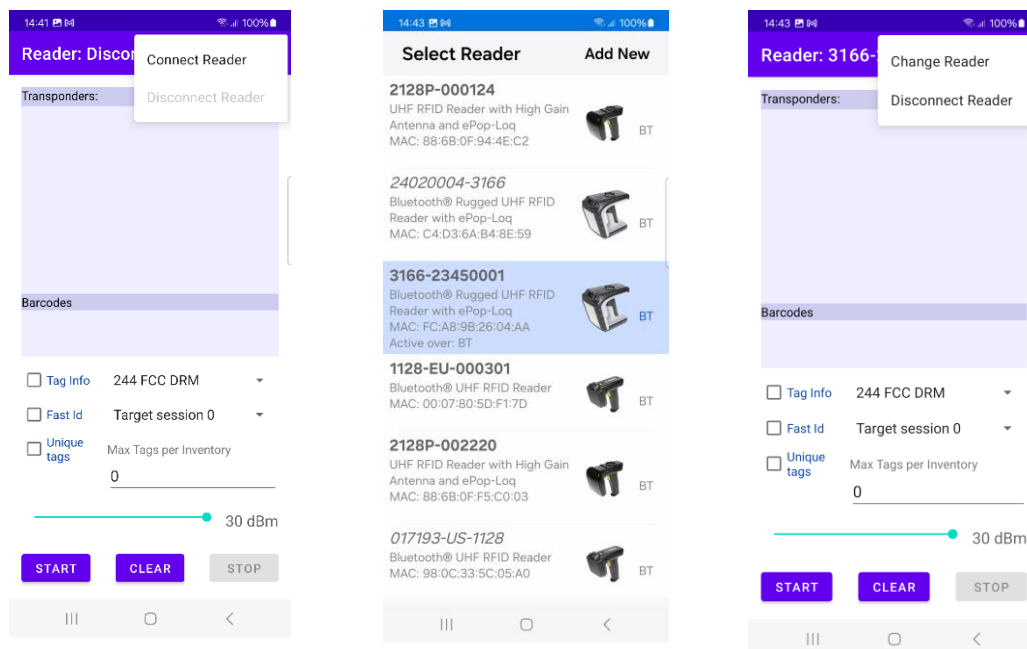


Figure 2 The common reader menu and the DeviceList Activity

The sample applications shipping with the SDK are all built using the same basic template which provides the reader connection, disconnection and reset operations from the action menu. The operations available are:

- [Connect Reader/Change Reader](#) – Launches the [DeviceList](#) Activity that provides a “Select Reader” interface for Users to choose the TSL Reader to use.
- [Disconnect Reader](#) – Disconnects from the currently connected Reader.
- [Reset Reader](#) – Resets the currently connected reader to the factory default settings.

The samples all use the sharedInstance of [AsciiCommander](#), as we (currently) recommend so there is a call to `createSharedInstance()` in the `onCreate()` method of the launch Activity.

The main [Activity](#) for all sample projects provides the default configuration of the [AsciiCommander](#) to include a [LoggerResponder](#) and a [SynchronousResponder](#) in the `OnCreate()` method. The [Activity](#) then creates the model which provides any additional responders as needed (often through an `initialise()` method or similar).

The reader specific operations are separated out into a model class derived from [ModelBase](#). The `ModelBase` class uses a `Handler` property to communicate with the UI thread and provides a `performTask(Runnable task)` method to allow long running operations (including synchronous ASCII commands) to be executed on a non-UI thread. A `Busy` property is used while `performTask()` runs to signal via the `Handler` when the Task is active or not. A [ModelException](#) is provided to indicate failure of a Task.

Inventory Sample Application

The Inventory sample project demonstrates some of the operations available using the [InventoryCommand](#). It also illustrates use of the [BarcodeCommand](#) as a Responder for capturing barcodes from the reader.

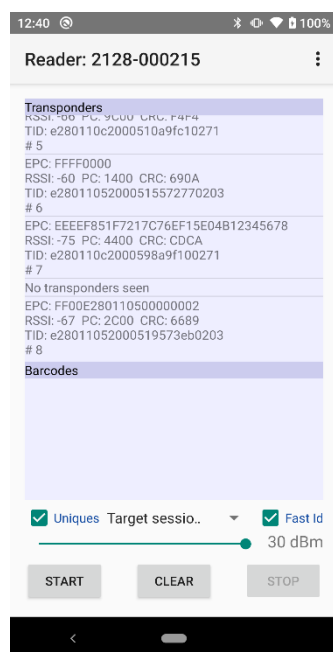


Figure 3 The Inventory sample application.

The Inventory application uses the default action of the device trigger to produce inventories on single press and barcode scanning from the double press action.

A pop-up menu can be used to configure the session used for inventories and the slider bar can be used to adjust the antenna output power.

Scanned tags will appear in the upper list area and for each transponder will contain information about the signal strength, the PC control word and the CRC value. If the tag supports it¹ and the [FastId](#) checkbox is checked then the information from the TID memory will also be displayed.

If the [Uniques](#) checkbox is checked then each tag will appear only once in the list (if the list was clear before the box was checked)

Barcode scans appear in the lower display area preceded by the prefix 'BC:'.

Inventory Sample Code Overview

The [InventoryModel](#) uses two instances of [InventoryCommand](#), one ([mInventoryCommand](#)) is used to issue inventory commands and configure the reader and the other ([mInventoryResponder](#)) is used as a [Responder](#) to capture inventory responses. A [BarcodeCommand](#) is also used in the role of [Responder](#) to capture incoming barcode scans.

The model adds and removes the responders from the current [AsciiCommander](#) to enable/disable (respectively) the reporting of scanned transponders/barcodes to the [InventoryActivity](#) via the model's [Handler](#).

¹ For example, Impinj M730 Transponders

The inventory responder sets an *[ITransponderReceivedDelegate](#)* to handle each of the transponders received, extracts some of the returned information and uses the model's *[sendMessageNotification\(\)](#)* method to present this to the UI. The barcode responses are handled in a similar manner by the barcode responder.

The model uses the *[InventoryCommand's ResponseLifecycleDelegate](#)* to control the continuous inventory. In the *[responseEnded](#)* method, the code checks a flag, *[mContinuousScanEnabled](#)*, and when true, issues another Asynchronous *[InventoryCommand](#)*. This method also contains code to play an alert sound when tags are seen – it does this only if sufficient time has passed, since the last alert sound, to avoid the sound being continuous on 11xx series readers.

The *[InventoryModel's updateConfiguration\(\)](#)* method uses the *[InventoryCommand's TakeNoAction](#)* property to ensure that the trigger-initiated inventory matches that of the UI Scan button. As the session, output power or fast id setting changes in the UI the *[updateConfiguration\(\)](#)* method is called and the reader's inventory (.iv) command parameters are set without performing an (inventory) action.

The *[InventoryActivity](#)* also demonstrates the use of the *[AsciiCommander getDeviceProperties\(\)](#)* method. The returned *[DeviceProperties](#)* instance contains information about the device that differs from the default ASCII protocol such as the antenna power limits. This is used to adjust the limits of the UI's power level trackbar when the connected reader changes e.g. maximum of 25 dBm for an 1153 reader or 29 dBm for the 1128 reader.

Trigger Sample Application

The Trigger sample project demonstrates monitoring and responding to the change of state of the trigger switch. It also demonstrates changing of the default trigger behaviour.

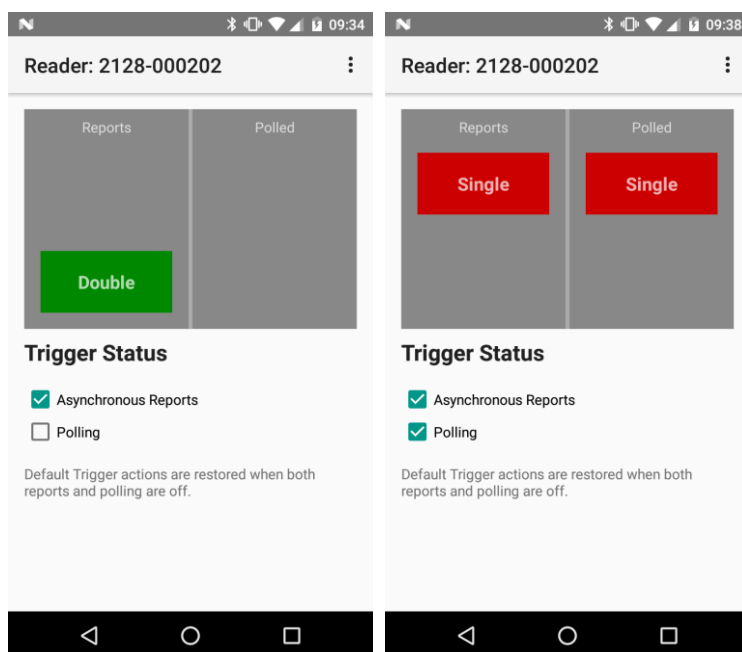


Figure 4 The Trigger sample application.

The Trigger application provides an on-screen display of the state (Single, Double or Off) of the device trigger switch and determines the switch state using two methods:

- asynchronous switch state reports
- continuous polling of the switch state

Asynchronous reports are also indicated by an accompanying beep from the device.

When any of the trigger status options are selected the default actions of the trigger are disabled. When both of the monitoring options are disabled the default actions of the trigger are restored.

Trigger Sample Code Overview

The Trigger application reader specific code is in the [TriggerModel](#) which determines changes of switch state using two methods:

- Via asynchronous switch state reports, set with the [SwitchActionCommand](#)
- By polling the switch state using the [SwitchStateCommand](#).

The [TriggerModel](#) provides two properties to control the switch monitoring [AsyncReportingEnabled](#) and [PolledReportingEnabled](#).

When either monitoring method is enabled the [SwitchActionCommand](#) is used to change the default trigger switch behaviour to do nothing. This command is also used to enable the asynchronous reporting when needed.

The [TriggerModel](#) base class messaging mechanism is extended to provide unique notifications to the UI for asynchronous and polled switch state change notifications.

For monitoring asynchronous reports an instance of [SwitchResponder](#) is used which captures the reader's switch changed notifications and passes them to its [ISwitchStateReceivedDelegate](#)'s [switchStateReceived\(\)](#) method. When

the `switchStateReceived()` method is executed the UI is notified using the model's message system and an (asynchronous) `AlertCommand` is configured and executed to provide audible feedback of the switch state. This responder is added to the current commander's responder chain from the model's `initialise()` method.

The polling method uses the model's `Handler` to execute a `Runnable` periodically. The `Runnable` instance uses a synchronous `SwitchStateCommand` to interrogate the reader for the current switch state. When the command completes the `SwitchStateCommand`'s `State` property holds the trigger state which is passed to the UI via the model message system. Note that this approach to periodic polling causes the `SwitchStateCommand` to execute on the UI thread and, while the command will usually execute very quickly, this may introduce some lag to the UI responses.

ReadWrite Sample Application

The ReadWrite sample project demonstrates reading from and writing to transponders. The commands can operate on one or more transponders simultaneously depending on the EPC specified.

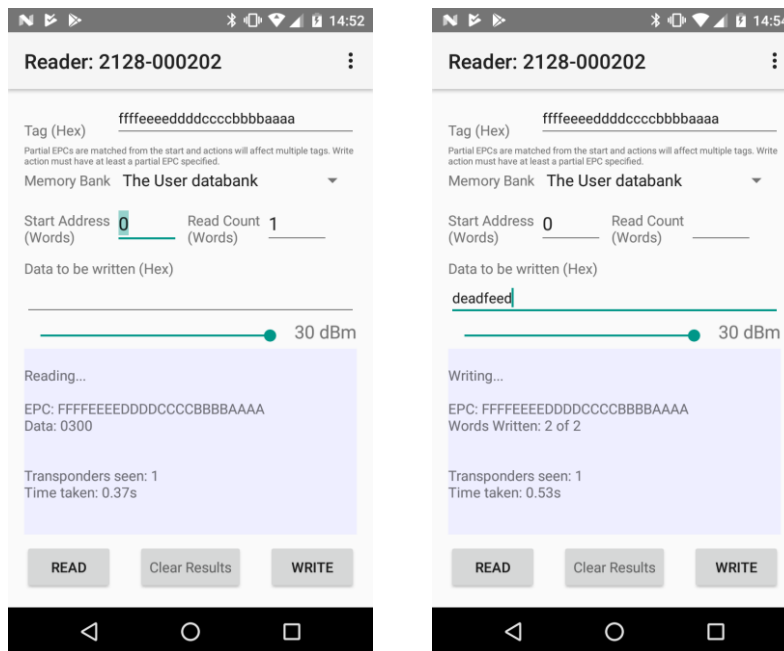


Figure 5 The ReadWrite sample application – Reading (left) - Writing (right)

The ReadWrite application provides an interface for configuring and executing operations that demonstrate the use of the [ReadTransponderCommand](#) and [WriteTransponderCommands](#).

The interface fields and their usage are described below:

Tag (Hex) – Enter either a full or partial EPC here. Partial EPCs are matched from the start of the EPC. As a safety precaution, Write commands must have at least a partial value entered.

Memory Bank – Select the memory bank for the operation using the pop-up menu.

Start Address – Specify the starting offset into the chosen memory bank in words.

Read Count – Specify the number of words to be read here.

Data to be written – Enter the data to be written here as a hex value.

Power Slider – Adjust the Antenna power using the slider as desired.

Once the appropriate configuration has been set, the **Read** or **Write** buttons will execute the command.

The outcome of the command is reported in the display area above the buttons. This area can be cleared using the **Clear** button.

ReadWrite Sample Code Overview

The ReadWrite application's reader-specific code is in the [ReadWriteModel](#) which provides access to both a [ReadTransponderCommand](#) and a [WriteTransponderCommand](#). Changes to the UI field values are used to change the appropriate command properties.

Pressing the **Read** or **Write** button will cause the appropriate command's [ISelectMaskParameters](#) properties to be set for the given EPC. The [ISelectControlParameters](#) and the [IQueryParameters](#) properties are set to use a session with a long persistence time and to select the target tags into the B state if an EPC is specified otherwise (for read command only) an inventory only of session 0, state A is specified.

Each command uses the *ITransponderReceivedDelegate* to report progress via the Model's messaging system. An external field is used to count the number of transponders seen. Note that the delegate will never be called if no transponders respond.

Since there are potentially many transponders that could respond to any given operation the *ModelBase.PerformTask()* method is used to execute the commands on a separate thread. This will send notification of the *Busy* property changes via the model's messaging system. As the operations are using synchronous commands the model will be 'busy' for the duration of the read or write.

Upon completion of the execution of the command the *reportErrors()* method is used to detect any command configuration errors and forward them to the UI's result area. Note that the *isSuccessful* command property will be false when the command does not report any transponders.

Licence Key Sample Applications

The Licence Key sample code consists of two applications:

- **Licence Key** – This 'Installer' app demonstrates setting and deleting a licence key from a reader using the [LicenceKeyCommand](#) which is available in [ASCII Protocol 2.2](#) and above.
- **LicenceKeyUserApp** – This app shows that the addition of a licence key to the reader does not affect how the reader operates it simply provides a method for software Apps to only respond to *authorised* readers.

These apps replace the single app from previous SDK releases and are intended to show how the [LicenceKeyCommand](#) can be used to manage a Vendor's 'complete' hardware and software RFID solution where only Vendor-authorised Readers can be used with the Vendor's software.

In this release, the type of licence key used is an asymmetric, elliptic-curve, digital signature, generated from unique Reader characteristics and using the standard Java/Android cryptography libraries. This approach uses a public/private key pair where the public key is used for verification and the private key is used for authorisation. In this arrangement, only the Private key needs to be kept secure and under the Vendor's control. The public key, embedded in the Vendor's publicly facing software, can only be used for verification and cannot be used to generate valid licence keys.

Warning: DO NOT USE THE SAMPLE KEYS PROVIDED IN PRODUCTION SOFTWARE! Also, this sample code is implemented, primarily, for developer understanding and does not represent "industry best practice" for managing (private) keys. Secure handling of the private key is the responsibility of the developer/implementer.

LicenceKey

The LicenceKey app demonstrates how a Vendor can *Authorise* (or *Enrol*) Readers for use with their software by installing a [licence key](#) on the device.

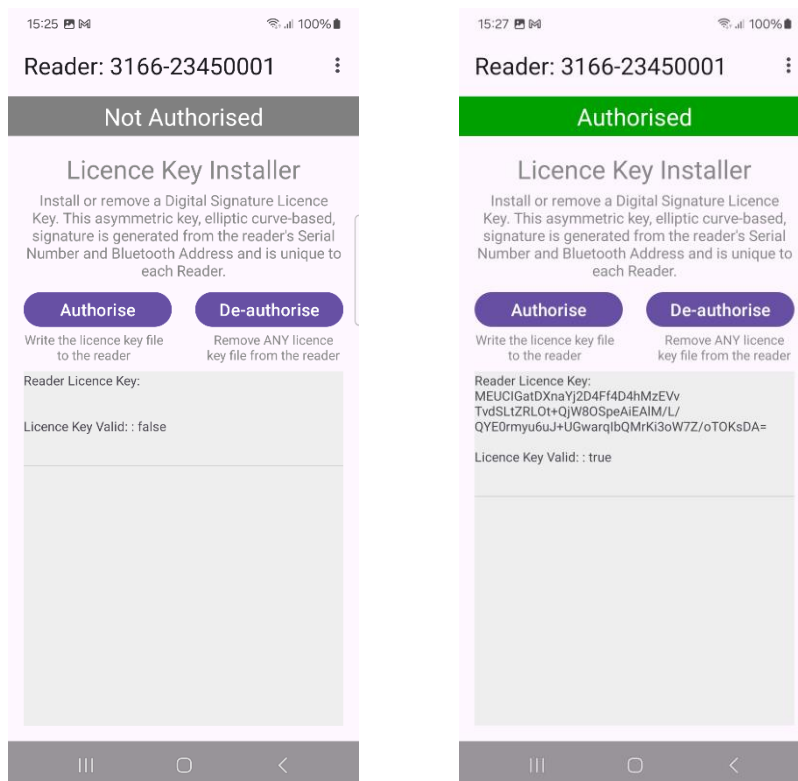


Figure 6 the LicenceKey App – Before a Reader has been authorised (left) and after (right)

The interface fields and their usage are described below:

Authorisation Indicator – At the top of the screen is an indicator that will show the authorisation state of the currently connected reader. An *authorised* reader is a reader that has a licence key generated from the device's serial number and the device's Bluetooth address. The authorisation indicator is grey for non-authorised devices and green if the reader is authorized.

Authorise Button – Tap this button to write a licence key to the current reader.

De-authorise Button – Tap this button to remove ANY licence key from the current reader.

Various messages are shown in the grey area at the bottom of the screen.

LicenceKeyUserApp

The LicenceKeyUserApp represents the Vendor's software offering which can test a connected Reader for the presence of the [licence key](#) and make the software respond accordingly. The Vendor's software may choose to restrict the feature set to a 'demo' or 'limited functionality' mode or may choose to not respond to unauthorised readers at all!

In this demo code, how to respond to the connected Reader is selectable by an on-screen control to emphasize that it is the software that decides the app's functionality based on the presence or absence of the (valid) [licence key](#).

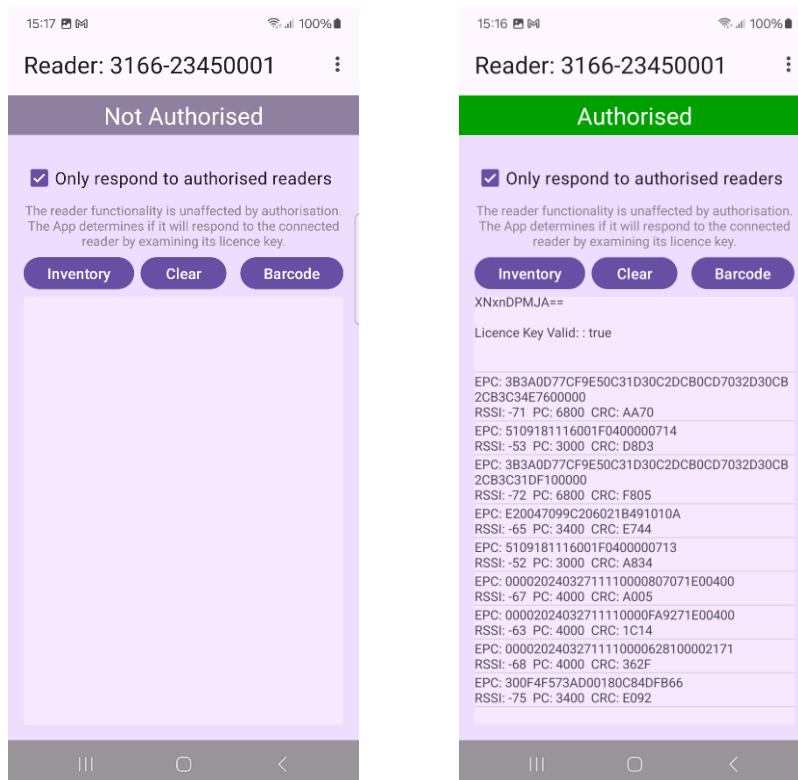


Figure 7 the LicenceKeyUserApp – with an unauthorised Reader (left) and an authorised Reader (right)

The interface fields and their usage are described below:

Authorisation Indicator – At the top of the screen is an indicator that will show the authorisation state of the currently connected reader. An [authorised](#) reader is a reader that has a licence key generated from the current [Secret](#), the device's serial number and the device's Bluetooth address. The authorisation indicator is grey for non-authorised devices and green if the reader is authorized.

Only respond to authorised readers – When this is checked the [Inventory](#) and [Barcode](#) operations will only function if the current reader has a valid licence key. If it is unchecked, then the [Inventory](#) and [Barcode](#) actions works as expected irrespective of the readers licence key (if any). This check box also applies to responses generated using the Reader's trigger.

Inventory Button – Tap this button to perform an inventory of any tags in range.

Barcode Button – Tap this button to initiate a barcode scan.

Clear Button – Tap this button to clear the results area.

Various messages are shown in the area at the bottom of the screen.

Licence Key Sample Code Overview

LicenceKey

The Licence Key application's reader-specific code is in the [LicenceKeyModel](#) which uses the [LicenceKeyCommand](#) to manipulate the licence key stored on the currently connected reader.

The [LicenceKeyModel.authoriseReader\(\)](#) method uses the [LicenceKeyCommand](#) to delete any existing key and to write the new value generated by the [createLicenceKey\(\)](#) method. The [LicenceKeyCommand](#) can only write a licence key when the existing key is blank so the [setDeleteLicenceKey\(\)](#) property is used to achieve this.

The [createLicenceKey\(\)](#) method uses the Signature class, from the standard Android/Java cryptography libraries, to generate the licence key using the Private key and a 'message' string formed from the Reader's serial number and Bluetooth MAC address.

The [validateReader\(\)](#) method sets the model's [isReaderAuthorised\(\)](#) property by using the Public key to verify the licence key read from the reader. The reader's serial number and its Bluetooth address are obtained using the [VersionInformationCommand](#), which has a [getBluetoothAddress\(\)](#) method (for Readers implementing ASCII protocol 2.2 or greater). The two values are used to construct the same 'message' used by [createLicenceKey\(\)](#) and this message is used to verify that the signature in the licence key is for this, specific Reader.

The [deAuthoriseReader\(\)](#) method uses the [LicenceKeyCommand](#) command to simply delete any existing licence key.

LicenceKeyUserApp

The application's reader-specific code is in the [LicenceKeyUserAppModel](#).

The [validateReader\(\)](#) method is identical to that in the LicenceKeyModel - this code only requires the Public key to implement the verification and this key cannot generate valid licence keys!

The remainder of the code shows how an App may make use of the [isReaderAuthorised\(\)](#) property to restrict the options available within the App. In this App, the [isOnlyAuthorisedReaderAllowed\(\)](#) property also allows this [restricted](#) behaviour to be switched on and off for demonstration purposes.

The code also demonstrates the use of the [VersionInformationCommand](#), the [InventoryCommand](#) and the [BarcodeCommand](#).

ConfigureBarcode Sample Application

The [ConfigureBarcode](#) sample application demonstrates the commands available for modifying the behaviour of the TSL Reader's barcode engine (if fitted). The sample project provides UI elements to modify some of the options of the Code 128 symbology recognition.

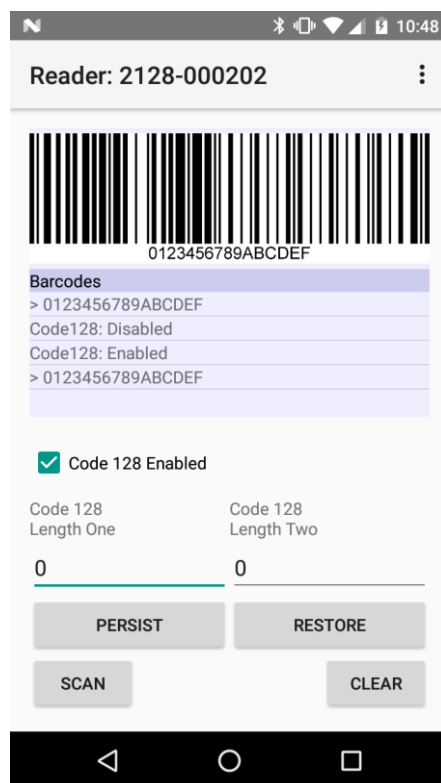


Figure 8 the ConfigureBarcode Sample Project

The interface fields and their usage are described below:

Code 128 Test Barcode – At the top of the screen is a Code 128 barcode containing the digits “0123456789ABCDEF”– if your terminal has sufficient resolution, you can use this to test the changes to the symbology parameters.

Barcodes – Under the test barcode is a results area showing the outcome of parameter modifications and barcode scanning.

Code 128 Enabled – When the check box is ticked the barcode engine will read Code 128 barcodes – when unticked the Code 128 symbology is disabled and the barcode scanner will fail to read any codes of this type.

Code 128 Length One – Enter values for the Length One parameter here.

Code 128 Length Two – Enter values for the Length Two parameter here.

Persist Button – Tap this button to make the current changes persistent across Reader power cycles.

Restore Button – Tap this button to restore the default settings for the Code 128 symbology.

Scan Button – Tap this button to initiate a barcode scan.

Clear Button – Tap this button to clear the results area.

ConfigureBarcode Sample Code Overview

The Configure Barcode application's reader-specific code is in the [ConfigureBarcodeModel](#) which uses the [Code128Symbology](#) class from the [BarcodeConfiguration](#) API to change the parameters of the symbology. The code also demonstrates the use of the [BarcodeCommand](#).

The [Code128Symbology](#) instance is instantiated using the current AsciiCommander as all parameter changes are propagated to the Reader dynamically. So, if many configuration changes are to be made then ensure that this occurs on a background thread.

Currently only Motorola barcode engines are supported - the SDK Javadocs provide details of the parameters and their valid values.

Use of these classes can result in persistent changes to the Readers barcode scanning capabilities – we highly recommend that developers provide Users with clear warnings of the persistent nature of any changes. Alternatively, direct Users to use the [TSL Reader Configuration](#) App to make these changes.

ReaderManager Sample Application

The Reader Manager sample application is a new code sample built upon modern App foundations and demonstrates the use of the [ReaderManager](#) and [Reader](#) classes to connect to TSL ASCII 2 Readers. Its purpose is to demonstrate the connecting of a Reader and so, provides minimal RFID functionality, which is used only to verify that a Reader has connected successfully.



Figure 9 the ReaderManager Sample Project

The interface fields and their usage are described below:

Menu – The menu at the top right of the screen provides items to Connect and Disconnect from a TSL ASCII 2 Reader.

Message Area – Most of the UI is a message area that displays the outcome of connection attempts, Activity lifecycle events and incoming Reader data (when connected).

Action button – Clears the message area.

ReaderManager Sample Code Overview

The Reader Manager application's creation is described in the accompanying document "[Creating the ReaderManager Sample Project](#)". This is a step-by-step guide that starts from a new Android Basic Project and results in the Reader Manager sample project.

See the above section "Connecting to TSL UHF RFID Readers" for a description of the classes and objects used in the sample project.

TagFinder Sample Application

The TagFinder sample application demonstrates the use of the *FindTagCommand* (Series 21xx Readers or later) or the *InventoryCommand* (Series 11xx Readers) to obtain the signal strength (RSSI) from tags with EPC's matching a User-defined pattern.

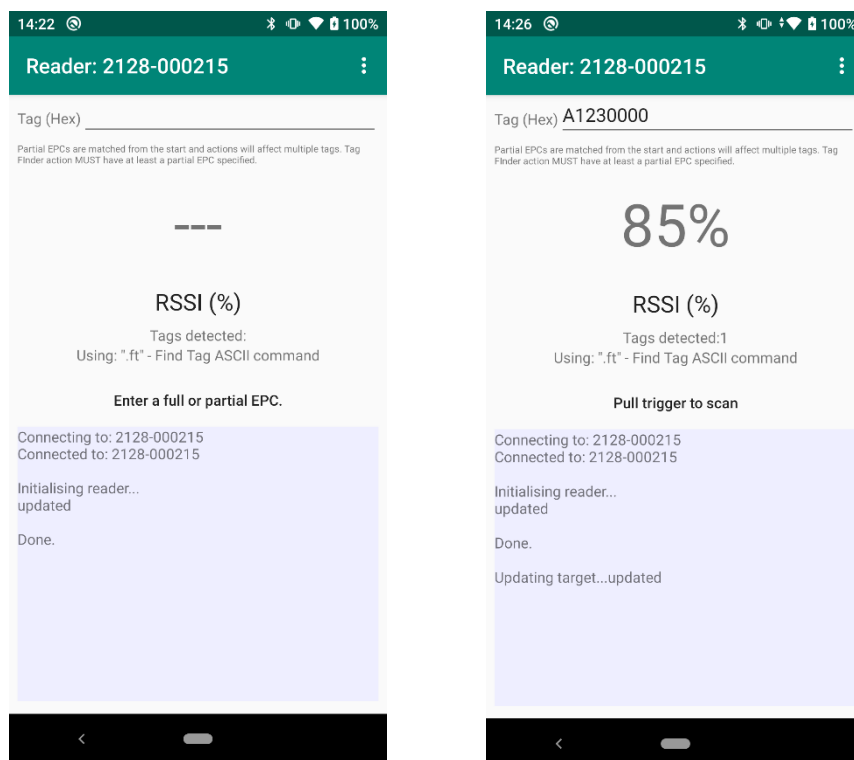


Figure 10 the FindTag Sample Project

Once a valid (full or partial) EPC has been entered, pulling and holding the Reader's trigger will produce a sequence of percentage values for the RSSI of the target tag. For series 21xx readers there will also be audible tones representing the signal strength.

The interface fields and their usage are described below:

Tag(Hex) – At the top of the screen is a field where the full or partial EPC of the target tag is entered. Ensure that the keyboard's return/next field button is pressed when data entry is complete to indicate that the text has changed. When the value of this field has changed, the Reader will be updated to target the new EPC.

RSSI(%) – This value is the current RSSI value returned for the target tag as a percentage. Its value will vary as the distance (or orientation) between the Reader and the tag changes.

Tags Detected – This field will indicate how many tags matched the target EPC value.

Using "..." – This field indicates which ASCII 2 command is being used to obtain RSSI values. For 21xx Readers (or later) this will be the FindTag command otherwise the Inventory command is used.

"Pull Trigger to scan" etc... – Instructional messages are displayed to guide the user.

Message area – At the bottom of the screen is a message area used to display additional information.

Tag Finder Sample Code Overview

The TagFinder application's reader-specific code is in the *TagFinderModel* which uses either the *FindTagCommand* or the *InventoryCommand* to obtain RSSI values, depending on the currently connected Reader.

Each time a Reader connects, the [TagFinderActivity](#) calls the model's [resetDevice\(\)](#) method. This method attempts to execute the [FindTagCommand](#) with the [TakeNoAction](#) flag set, and, if successful, will indicate that the [FindTagCommand](#) should be used to obtain RSSI values. If unsuccessful then the [InventoryCommand\(\)](#) is used.

Whenever the User changes the target Tag EPC (or a Reader connects) then [the updateTargetParameters\(\)](#) method is called. This method has two purposes 1) set up the trigger to issue both asynchronous notifications and the appropriate command (as determined by the [resetDevice\(\)](#) method) and 2) to configure the chosen command ([FindTagCommand](#) or [InventoryCommand](#)) to target the tag specified in the [Tag\(Hex\)](#) field.

The model uses a [SignalStrengthResponder](#) to capture the RSSI value from whichever command is in use. It provides the [ISignalStrengthReceivedDelegate](#) which is used by the [FindTagActivity](#) to receive the RSSI percentage values obtained. It also provides the [ISignalStrengthCountDelegate](#) to indicate how many tags matched the target pattern.

A [SwitchResponder](#) is used to capture the asynchronous switch state notifications which are used to clear the RSSI value when the Trigger is released.

HFLFSupport Sample Application

The HFLFSupport sample application shows the capabilities of the ASCII Protocol commands targeted at HF/LF Readers. The default trigger action is used to scan for tags or read barcodes and these values are reported to the message area. The app is configured so that HF/LF tags also report their type, which is displayed along with the tag identifier.

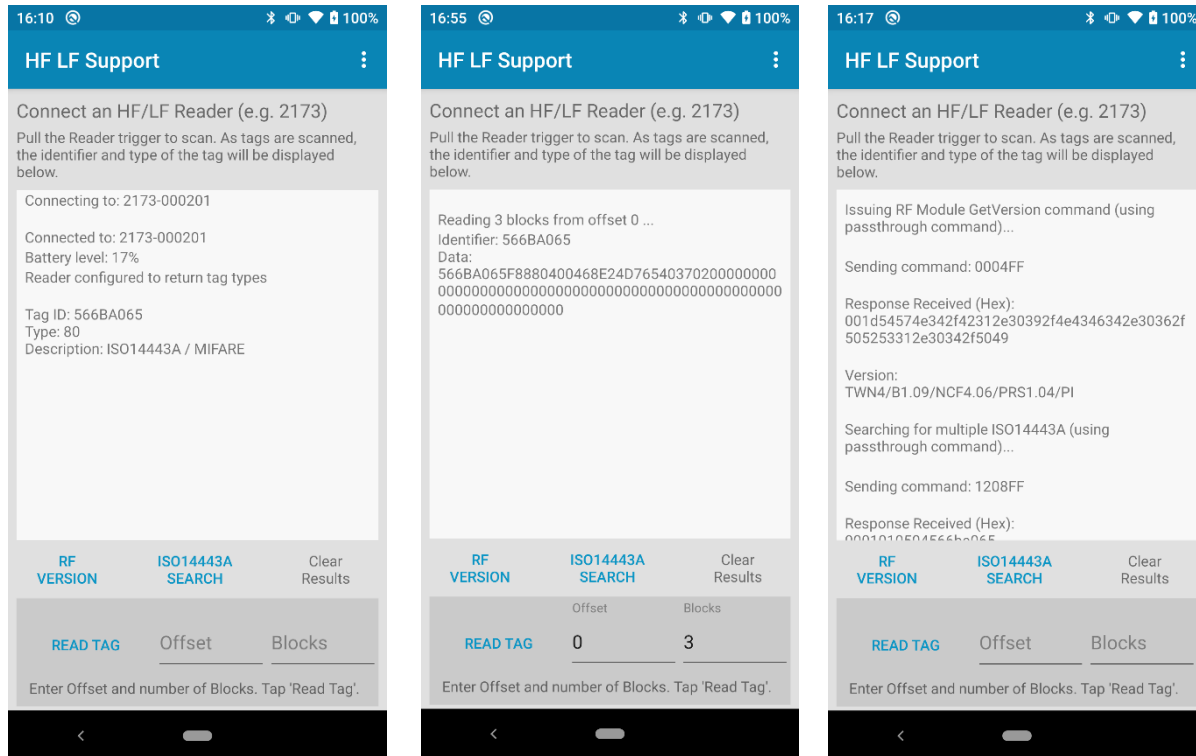


Figure 11 the HFLFSupport Sample Project

The memory of a tag can also be read by entering the offset to the first block and the number of blocks to read into the UI and tapping the **READ TAG** button

The interface fields and their usage are described below:

Instructions – At the top of the screen is a message describing the basic scanning operation.

Message area – The message area is used to display all responses from executed commands and scanned tags.

RF VERSION button – Tapping this button retrieves the Reader's RF Module version.

ISO14443A SEARCH button – Tapping this button will perform a search for multiple ISO14443A tags, reporting each as they are found.

Clear Results button – Tap this to clear the Message Area.

READ TAG button – Tap this after entering appropriate Offset and Number of Blocks values. This will execute a read of the tag memory from *Mifare Classic*, *ISO15693* and *SRX* tags.

HF LF Support Sample Code Overview

The HFLFSupport application's reader-specific code is in the *HFLFSupportModel* which uses the *InventoryTagCommand*, the *ReadTagCommand* and the *RfPassThroughCommand* to obtain identifiers and data from HF and/or LF Tags.

Each time a Reader connects, the *HFLFSupportActivity* calls the model's *resetDevice()* method. This method executes the *InventoryTagCommand* with the *TakeNoAction* flag set, to configure the Reader to return Tag types. It also queries the battery level using a *BatteryStatusCommand*.

The model uses an [InventoryTagCommand](#) as a responder, with the [CaptureNonLibraryResponses](#) property set to true, to capture tags resulting from Trigger presses. Each transponder found is processed by the [TransponderReceivedDelegate](#) to format and dispatch identifier and other information to the [HFLFSupportActivity](#) for display in the message area. This responder is added to the AsciiCommander's responder chain when the model is enabled and removed when the model is disabled. A [BarcodeCommand](#) is used in a similar way to handle scanned barcodes.

The [ReadTagCommand](#) is executed as an [AsyncTask](#) when the **READ TAG** button is tapped. To support Mifare tag access, there are a set of parameters to specify the Mifare key type (A/B), the corresponding (A/B) password and the Sector to be accessed. These properties are currently hard-coded with typical values for new Mifare tags – change these values to match your tag's specific encoding. Each transponder found is processed by the [TransponderReceivedDelegate](#) to format and dispatch identifier and other information to the [HFLFSupportActivity](#) for display in the message area.

The HF/LF module is capable of accessing a large range of tags, many with their own proprietary formats. These are currently supported by using the [RfPassThroughCommand](#) and the [TWN4 Simple protocol](#). This command allows the execution of any command described in the [TWN4 Pass Through Mode](#) document.

Two such commands are demonstrated in the App:

- 1) [GetVersion](#) – to retrieve the HF/LF module version.
- 2) [ISO14443A_SearchMultiTag](#) to scan for multiple tags simultaneously.

The [RfPassThroughCommand](#) has a [CommandData](#) property to hold the byte[] representing the command to be executed and, after execution, the [ResponseData](#) property will contain the byte[] that is the response to the command. The encoding and decoding of these byte arrays is described in the [TWN4 Pass Through Mode](#) document.

S3SensorTagReader Sample Application

The S3SensorTagReader sample application demonstrates access to the [Sensor](#), [On-Chip RSSI](#) and [Temperature Code](#) values from transponders that are based on the Magnus® S3 RFID chip, such as the [HID Sense Passive](#) range or the Axzon sensor tags.

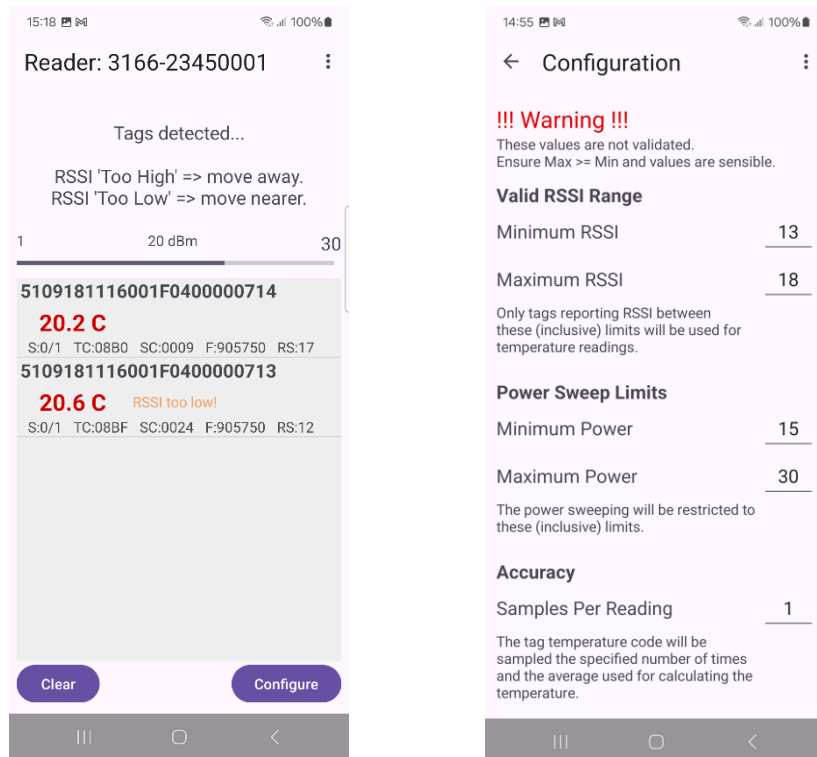


Figure 12 The S3SensorTagReader Sample Project - Main Screen (left) and Configuration Screen (right)

While all [TSL RAIN RFID Readers](#) are capable of reading the [Sensor](#) and [On-Chip RSSI](#) values, **access to the Temperature Code values requires a TSL Series 3 UHF RAIN RFID Reader**, such as: [3166 Bluetooth® Rugged RAIN® RFID UHF Reader](#).

The interface fields (for the main screen) and their usage are described below:

Message area – At the top of the screen is a message area used to display additional information.

Power Level Indicator – This displays the Reader's current power level.

Transponder List Area – Any transponders discovered will be displayed here along with the various sensor values that have been read. See below for further details.

Clear – Resets the list of sensor readings.

Configure – Navigates to the [Configuration](#) page where parameters used in the tag access can be modified.

Basic Operation

Ensure that a *Series 3* Reader is connected. If an incompatible Reader is connected, then the app navigates to a separate warning screen.

With some S3 sensor tags nearby, pull the trigger on the Reader: once to *start scanning* and then once more to *stop scanning*.

When a scan starts the following occurs:

- A scan for 'valid' transponders, with on-chip RSSI in correct range, is made. The valid RSSI range is specified on the *Configuration* page and defaults to levels specified in the [Axzon AN006 Application Note](#) to achieve maximum accuracy for temperature reading.
 - As recommended, the power sweep starts at a low level to avoid overloading the sensors.

- Any transponders detected will cause a *Sensor Reading* item to appear in the list.
- If no 'valid' transponders are found
 - The power level is adjusted based on the On-Chip RSSI response of any tags seen – raising the power when RSSI is too low and lowering power if RSSI is too high.
 - If no tags are seen, then power level is raised – ensure that the sensor tags are not too far away.
 - Power is varied between the limits set on the *Configuration* page.
- If 'valid' transponders are found
 - Power is adjusted to the maximum level possible while keeping the tags in the valid RSSI range.
 - For each transponder found:
 - Calibration data, for the conversion of Temperature code to Celsius is retrieved.
 - The first time a tag is seen, the calibration data is read from the tag see [Axzon AN002 Application Note](#) for details.
 - The calibration data is cached and retrieved when the tag is seen again.
 - A read of the tag's *Temperature Code* is made and recorded.
 - If sufficient samples have been retrieved, then the average of the samples is converted to Celsius and displayed.
 - The number of samples per reading is specified on the *Configuration* screen.
 - [Axzon AN006 Application Note](#) gives details of the accuracy that can be expected for different numbers of samples.
 - Temperature Code*, *Sensor Code*, *Channel Frequency* and *RSSI* values are displayed for any tags seen.
 - Tag-specific messages may also be displayed e.g. RSSI out of range.

While scanning, hold the Reader in a fixed position until the power sweep locates the tag (or tags). Once the power sweep has settled, small adjustments of the reader to tag distance *may* be needed to maximise the read rate or to bring multiple tags into reading range.

The list of sensor readings can be cleared by tapping the *Clear* button.

Tapping the *Configure* button will navigate to the *Configuration* screen where certain parameters, used in the above algorithm, can be adjusted. Accessing the *Configuration* screen will stop any scanning. For simplicity, in the sample code, it is assumed that parameters *will* be changed and so the list of sensor readings is also cleared.

Sensor Reading Display Area

Each tag detected is displayed in the list as shown below:

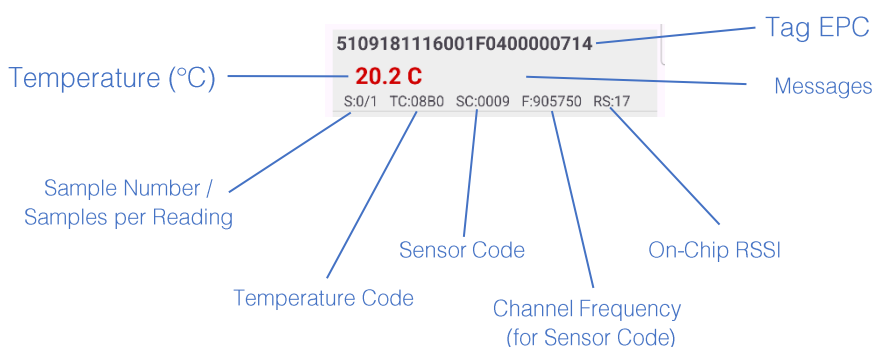


Figure 13 The Sensor Tag Reading display.

This Sensor Reading display will update as the transponder is read – it will fade slightly when a tag has not been seen for several scans.

S3SensorTagReader Sample Code Overview

This sample project is considerably more complex than the other projects to demonstrate all the steps needed to access all of the values from the sensor chip.

The `S3SensorTagReader` application's reader-specific code is in the `SensorTagModel` which uses several `ReadTransponderCommands` to obtain the sensor values and calibration data. The model also uses the `SwitchActionCommand` to enable asynchronous switch notifications and disable the default trigger actions. A `SwitchResponder` is used to monitor the change of state of the trigger and start and stop the scanning accordingly.

Because the sample app disables the Reader's standard trigger behaviour, the Reader is reset to defaults whenever the App is paused (using a `FactoryDefaultsCommand`).

Each time a Reader connects, the `SensorTagFragment` calls the model's `configureReader()` method. This method modifies the trigger behaviour, as described above. This method is also called from the `SensorTagFragment`'s `onResume()` method to ensure that when the App resumes the Reader is ready.

The model has an enabled state property (`is/setEnabled()`) that is called from the `SensorTagFragment` as the User navigates between screens. Enabling the model causes the `SwitchResponder` to be added to or removed from the `AsciiCommander`, as appropriate, and also stops all scanning when disabled.

Whenever the `SwitchResponder` detects a Single press, it toggles the `isScanning` state and starts or stops the scanning as required. This is achieved using the two methods `startContinuousTemperatureReading()` and `stopContinuousTemperatureReading()`. The `startContinuousTemperatureReading()` method runs the scanning code (method `continuousTemperatureReading()`) as a background task (see `ModelBase` class).

The `continuousTemperatureReading()` method uses three `ReadTransponderCommands`:

`readRSSICommand`: this command accesses both the `Sensor Code` and the `On-Chip RSSI` using a single command, selecting for S3 tags with any `On-Chip RSSI` value (0-31). The channel frequency is also obtained however, this sample code simply reports this value but it could be used to compensate for the frequency-dependent variation in `Sensor Code` values (see [Axzon AN006 Application Note](#)).

`readTemperatureCommand`: this command is used to initiate and retrieve the temperature values. The Select parameters direct the tag to start a temperature sample.

The parameter `setSelectPause(3)` provides the recommended, additional continuous carrier wave described in [Axzon AN006 Application Note](#). This parameter is new to ASCII Protocol 3 and is only supported on Series 3 Readers.

- 1) `readCalCommand`: this command retrieves the factory-set calibration data (stored in User memory), which is used to create an instance of `MagnusS3TemperatureConverter` for the conversion to Celsius.

FYI Calibration data is verified using a CRC – the sample code uses the `CrcCalculator` class (MIT licence Copyright (c) 2017 Anton Isakov <http://crccalc.com>)

`TagReading` class instances are used to hold the tag specific `MagnusS3TemperatureConverter`, the various sensor data values and to average the temperature values according to the set number of samples per reading.

A simple loop, controlled by the `mlsRunning` flag controls the continuous scanning process. The flag is modified by the `start/stopContinuousTemperatureReading()` methods.

The power sweeping algorithm sets the `mCurrentPower` variable for use by the `readRSSICommand` and `readTemperatureCommand`. Reading of the calibration data is done at full power.

About TSL



Technology Solutions UK Ltd (TSL®), part of HID, is a leading manufacturer of high-performance mobile RFID readers used to identify and track products, assets, data or personnel.

For over two decades, TSL has delivered innovative data capture solutions to Fortune 500 companies around the world using a global network of distributors and system integrators. Specialist in-house teams design all aspects of the finished products and software ecosystems, including electronics, firmware, application development tools, RF design and injection mould tooling.

TSL is an ISO 9001:2015 certified company.



Contact

Address:	Technology Solutions (UK) Limited, Suite A, Loughborough Technology Centre, Epinal Way, Loughborough, Leicestershire, LE11 3GE, United Kingdom.
Telephone:	+44 1509 238248
Fax:	+44 1509 214144
Email:	enquiries@tsl.com
Website:	www.tsl.com

About HID



HID powers the trusted identities of the world's people, places and things. We make it possible for people to transact safely, work productively and travel freely. Our trusted identity solutions give **people** convenient access to physical and digital **places** and connect **things** that can be identified, verified and tracked digitally. Millions of people around the world use HID products and services to navigate their everyday lives, and billions of things are connected through HID technology. We work with governments, educational institutions, hospitals, financial institutions, industrial businesses and some of the most innovative companies on the planet. Headquartered in Austin, Texas, HID has over 4,000 employees worldwide and operates international offices that support more than 100 countries. HID is an ASSA ABLOY Group brand.

For more information, visit www.hidglobal.com.